System Requirements Analysis

🗞️ Summary & Key Points
‼️ Problem
♟️ Strategic Interests
💻 Product
💡 Product Insights
💥 Risks
🧭 Strategy
🏦 Cost
⚙️ Technical Strategy
- 💽 Core Back-Office Admin Panel
- ⏰ Auditability of Historical Records
- ✨ Consumer-Facing App(s)

🐾 Next Steps: Prepare the Project for Success
📚 Resources & References

🗞️ Summary & Key Points

There are risks to solve on two fronts:

- **Product strategy:** I strongly recommend working closely with a vendor that has good developers and understands Agile (or Lean!) product delivery to deliver the features “from day one” in small, finished increments. The project should focus on data integrity and modelling data correctly, first delivering an admin panel and extending that platform with custom applications.
- **Technical strategy:** I would recommend using Django + Django Rest Framework to create a custom back-office admin portal driven by well-defined data models over a CRM for a back-office admin panel, adding further custom functionality by adding custom consumer-facing apps to extend users’ interaction with that data. I would strongly recommend keeping the auditing and historical system simple for smaller numbers of records from previous experience working with both the mainstream alternatives, which are mostly

used in finance and could potentially double development costs due to complexity. Integrations will be a large part of the uncertainty, which should be isolated early on.

Cost: will vary greatly based on the specifics of each feature; however, a version could be 11-12 months by a *single* developer, given that a high amount of complexity is saved with the correct technical strategy for auditing and historical records. (see **Example translating time → money**)

‼️ Problem

The organisation is scaling quickly and needs a software system to sustainably support the integration of the core processes required for the smooth running of everyday business. Completing these tasks "offline" costs time and money from inefficient collaboration, with the added risk that data shared via email or an Excel sheet can be lost or compromised.

The Information System must be an interface between many internal and external users, allowing multiple teams to collaborate seamlessly in their functions between teams.

♟️ Strategic Interests

- **TTM (Time to Market):** every month the organisation don't have working software, they are losing time/money to manual tasks and building up a backlog of work which will need to be migrated to the new system. As the organisation expects to scale up quickly, an agile system that can be deployed quickly with short TTM becomes essential in adapting to changing requirements.
- **LTS (Long-Term Support):** support systems during and after development to maintain stability. It must be built on tried-and-tested technologies backed by large organisations with sufficient talent in the tech market to maintain the system in the long term.

💻 Product

- **Integrations:** <list-of-8-integrations>
- **Security:** AAD (Azure) authentication and authorisation across each product feature slice.
- **Auditability:** store and actively query select records at any version within their lifetime.
- **Reporting:** generate select PDF/ Excel documents from system data.
- **Alerting:** receive alerts when certain events are triggered
- **Back-office:**
  - **Users:** <list-of-6-user-groups>
  - <list-of-7-back-office-features>
- **Consumer-facing:**
  - **Users:** <list-of-3-user-groups>
  - <list-of-5-back-office-features>

💡 Product Insights

- **A suite of services:** the requirements describe multiple independently deployable services that can and should be broken down into “feature slices”, deployed incrementally, as deploying working software early allows:
  - **value to be delivered from day one**, allowing data to be fed into the system incrementally, preventing a backlog of data migration from building up while the system is still being built, and users get to use and learn the real system as usable features are added;
  - **promotes agility** in the software development lifecycle. Getting feedback from real users allows us to validate hypotheses about what brings value to users and course-correct when requirements need to be changed.
- **Data-driven:** the system depends on a **high level of data integrity** and should focus on **keeping clean data at the database level** (i.e. preventing invalid data states by using correct data modelling, which will eliminate an entire class of

possible data errors). Producing quality at this level flows into other aspects of the system.

- **Auditability:** should deliver auditability requirements **as simply as possible, whilst considering storage implications**, removing the complexities which can slowly destroy big projects by making development costs skyrocket to deliver simple features (such as backwards compatibility considerations with event-driven systems).
- **Security:** any offering should have authentication built in as a given, with horizontal and vertical authorisation policies included in the “definition of done” of a given feature (it’s not done until it's secured + GDPR compliant, etc.).
- **Uneven distribution of complexity:** a small pocket of certain features contributes disproportionately to the system's complexity. The order of work should reflect getting the best value for money, making the quickest wins that deliver the most value sooner and leaving things that are costly but deliver less value for later in the work order.

💥 Risks

▼ **Delivering zero value:** the development team doesn’t deploy the first feature immediately, waiting until “everything is ready”, only to deploy months later and find it’s not what the users wanted.

Mitigation: split up features into feature slices, which can be delivered incrementally to prevent the build-up of risk and deliver value from day one.

▼ **Complex auditability implementation:** the development team proposes a highly costly and complex auditability solution that over-engineers the requirements for the given scale. This leads to a longer implementation lead time, higher maintenance costs, and a shorter system lifetime.

Mitigation: a technical strategy that mitigates this complexity (see below).

🧭 Strategy

MVP-first, Lean software approach, splitting work into releasable feature slices to deliver value from day one. Features are delivered as they are developed in small chunks (e.g. first deliver a single feature where, for example, a system admin can assign users to a programme) frequently. The provider should work closely with the organisation to consult on the best practices for software development methodology and project management.

Phase 0 - Product Architecture Session [Initial work, revisited throughout delivery]

Goal: identify what "success" looks like for the organisation with the system, and connect features to the success of the project to inform prioritisation of work. Identify the users and how they interact with the system currently and in the ideal system.

Value: building a value structure within which the product team can work autonomously and identify smart ways to achieve success. This helps in building a plan to achieve success and clearly communicating to the developers what they are working on and why.

Phase 1 - Infrastructure Challenge [2-3 weeks]:

Goal: deploy infrastructure and environments for continuous delivery (CI/CD) and deploy the first feature to a production environment. Deploy a simple feature to this new infrastructure.

Value: every study of delivering value with software shows that Continuous Delivery is a key indicator of project success. By setting up deployment pipelines and deploying frequently, the development team has a reliable way to deploy and react to feedback from users quickly (see DORA metrics and *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High-Performing Technology Organisations*).

Phase 2 - De-Risk Core Third-Party Integrations [~1 week per integration]:

Goal: set up third-party integrations, authenticate, and set up a POC feature. The outcome is to identify further complexities/ work required on individual integrations.

Value: the biggest uncertainty in the project is the integration with other services. By creating a POC to clarify how data can be pulled into the system from each external service, risk factors are identified early.

Phase 3 - Develop Core Data Models and Admin Panel:

Goal: deliver a back-office admin panel where system admins can input data into the new system built around core data models. Identify features that can be developed quickly via an admin panel. Each feature built includes the required security/ auditability standards and third-party integrations.

Value: some features will deliver value through a generic back-office admin panel, which can be deployed very quickly and used to fulfil many user needs.

Phase 4 - Extend Features with Consumer-Facing App(s):

Goal: build frontend apps with more customisability depending on user demand and key business goals. Use third-party integrations where needed.

Value: basic MVP for data can be delivered through an admin panel, but where more customisation is required, it becomes more cost-effective and better user experience to build custom web apps - this is especially required for consumer-facing apps where branding is important.

Phase 5 - Iteration, Support, and Maintenance:

Continuously measure user experience, demand, changing business requirements etc. and react to changing requirements as priorities adjust to deliver the most important features.

🏦 Cost

The cost will depend on the size of the team, the experience of the team members, and the pricing model of whoever does the work (time and resources vs fixed costs for features). This must also be balanced with the required delivery time.

- **Team size:** smaller teams typically have less overhead and so can be more nimble and cost-effective, but lower capacity means spreading the cost over a longer period.
- **Team experience:** better developers go faster, especially where they have previous experience with the particular tech. Finding the right developers is very important as they can go orders of magnitude faster than an “average” dev (the “10x developer” - years of experience doesn’t equate to being a

better dev). Better devs also build more maintainable software, which means the solution lasts long (quicker build, lasts longer = significantly better value for money).

- **Pricing model:** <pricing-model-considerations>

Requirement estimations: the project's estimate may change as more details about the implementation are revealed through further feature refinement and product development.

That being said, taking the typical module to be Programme Management would take 1 dev 8 "dev weeks" → 8 x (~6 module tabs) ~ 48 dev weeks ~ 11-12 months for a *single* developer to develop the desired systems.

▼ **Example translating time → money**

<cost-modelling-example>

⚙️ Technical Strategy

💿 Core Back-Office Admin Panel

1️⃣ **Option 1 - Django + Django Rest Framework:** Used by Instagram, Spotify, YouTube, Google Search, Dropbox, Pinterest, and others to scale their products professionally.

✅ Advantages	⚠️ Drawbacks
• Admin panel support is built directly into the framework to generate a back-office portal. • Enterprise-ready (SQL) ORM built in to model and manage data migrations. • The plugin system offers (amongst others) a record versioning plugin for	• Uses Python, a dynamically-typed language which is highly accessible but has less "type-safety" than a language like TypeScript.

auditability and authorisation support for Azure AD.

- DRF creates REST API endpoints to be consumed by multiple apps quickly and flexibly.
- Open source software, the organisation have full control over data, data types, migrating away, and custom SQL queries.

2️⃣ **Option 2 - NestJS + AdminJS:** Used by IBM, GitLab, BMW, and others to scale their products professionally.

✅ Advantages	⚠️ Drawbacks
• Scalable design system suitable for large, complex enterprise apps. • Uses TypeScript (a single type-safe language shared across all services). • Not locked into the "Django way of doing things", easier to pick and choose third-party software (which makes it more customisable but easier to "go wrong"). • Open source software, the organisation have full control over data, data types, migrating away, and custom SQL queries.	• No record history plugin (comparable to Django) which satisfies the auditability requirements "out-of-the-box". • Fewer available features out-of-the-box; good for highly customised systems, but leads to the slower building of bespoke systems that don't need that level of customisation.

3️⃣ **Option 3 - SaaS CRM (Customer Relations Management):** e.g. Salesforce, HubSpot (I have built SaaS consumer products using HubSpot before)

✅ Advantages	⚠️ Drawbacks

- Allows for non-programmer configuration, allowing complex sales flows to be built with little to no programming for standard sales funnels.
- Often backed by a big company that can help support with the expertise required to use their systems in custom situations.
- Good document generation capabilities and email templating/ sending for notifications out of the box.
- Good for onboarding sales leads/ new people joining a new organisation and keeping track as they move through different application stages.

- **Data ownership + vendor lock-in:** CRMs keep your data on their infrastructure, making it difficult to access your data from a custom service and costly to migrate away.
- **Limited product features:** limited custom data types and data validation (customisation not required for most customers is usually offered through a complex backdoor). The majority of features built will require a custom interface to be built around the CRM to fill in the gaps
- **UX:** generic interfaces lead to a poor user experience if used as the primary
- **Target market:** more often tailored to pulling prospective sales leads through a sales funnel and tracking them through onboarding.
- **Cost:** will charge ongoing fees which you get locked into, and custom features require an “enterprise subscription” which can be £1000s and restricts the number of users.
- **Not version controlled:** can be misconfigured by users with permissions which can’t be reliably reset back to a stable condition with a push of a button.

❓ **Decision:** recommend Django + Django Rest Framework. It has a powerful admin panel built-in with many features that work out of the box quickly for development speed comparable to having a ready-made CRM. Django also comes with a smart solution to record version management which could be a pragmatic way to solve auditability requirements without overloading on complexity. Django's flexibility and talent market mean it is better for long-term support.
A CRM isn't versatile enough for the specified needs, so further custom systems must be built, adding complexity in designing the system to shoehorn a new service. Most of the technical complexity (auditability) isn't solved by the CRM, so it remains a problem that must be solved with custom systems built around the CRM. A CRM boosts speed where it meets *all* product needs, but in the long term, it slows the development of features.

⏰ Auditability of Historical Records

1️⃣ **Option 1 - Store Historical Records in a Relational Database**: store snapshots of records indexed by the time of update in a regular table.

✅ Advantages	⚠️ Drawbacks
• **Simple:** built-in to Django, MVP took 30 minutes to add to the project rather than months. • **Maintenance:** requires no expert knowledge to maintain well, so a wider pool of potential talent and lower cost to add/ change existing features. • **Low storage:** 10K records, each updated once per day → 1KB per record +20% for indexing → 12KB increase per day → 5GB storage per year	• **Data archiving:** as the system scales (after a few years), older archives will need to be archived to store space in the live database; querying older records is less frequent, therefore users don't mind waiting a slightly longer load time. • **Less specialised:** OLAP and Event Sourcing have their specialised use cases which perform better than this method when the core feature of the system is specifically to do with this feature.

2️⃣ **Option 2 - Event Sourcing Architecture**: store events of every change made to the system rather than the "actual state of data". Typically used with CQRS (command-query separation) e.g. I have worked with Axon Framework.

✅ Advantages	⚠️ Drawbacks
• Allows for a "replay" to occur to play through old events to rebuild the state of the database at any time. • Stores very granular details of changes to the system at every point.	• **Complexity:** a highly complex system which grinds development to a halt quickly as the system develops and requires internal backwards compatibility for every feature change. • **Performance:** Slows down over time as the event store grows. This can be somewhat mitigated by indexing columns and archiving short-lived records. • **Analytics:** OLAP is better for aggregating analytics. • **Expertise to maintain:** a niche knowledge base, and requires ~5 years of experience to master.

3️⃣ **Option 3 - Online analytical processing (OLAP) database**: a multi-dimensional database table (called a hypercube), e.g. I have worked with the IBM TM1 database.

✅ Advantages	⚠️ Drawbacks
• Very good for complex analytics cases (e.g. finance) where many fields need a "roll-up" to aggregate efficiently across multiple dimensions.	• **Cost of infrastructure:** due to more data being stored in multiple dimensions. • **Complexity of querying:** even simple queries must be disentangled from their dimensions.

- **Expertise to maintain:** a niche knowledge base, and requires ~5 years of experience to master.

❓ **Decision:** recommend storing the historical records in a relational database. Choosing the simplest option is a massive advantage for both time to market and long-term support of the system. Choosing either ES or OLAP leads to a system that can easily go wrong when developers don't understand the technologies or design implications leading to dead software projects (**this could be as much as doubling project costs**).

✨ Consumer-Facing App(s)

1️⃣ **Option 1 - React with NextJS:** backed by Meta and Vercel

2️⃣ **Option 2 - React with Remix:** backed by Meta and Shopify

3️⃣ **Option 3 - Angular:** backed by Google (less modern but opinionated framework)

4️⃣ **Option 4 - Emerging Frameworks (SvelteKit, SolidStart, QwickJS, Astro)**

5️⃣ **Option 5 - no/lo-code platforms (Webflow)**

❓ **Decision:** recommend NextJS (or Remix - similar but with some nuances depending on infrastructure) as it is a versatile declarative meta-framework backed by Meta and Vercel with a vast ecosystem of libraries to support responsive, full-featured modern web (and mobile) interfaces. Emerging frameworks offer some promising features (mostly focusing on performance, developer experience, and web standards); however, they are less tried-and-tested, and the talent market is smaller for developers to support the system long-term.
No/lo-code platforms build good non-interactive web pages designed to look good and be built fast, but provide much less flexibility very quickly.

🐾 Next Steps: Prepare the Project for Success

- Build out product architecture and value generation plan:
  - Build an understanding of the problem being solved
  - Build success criteria to align key stakeholders on what success looks like
  - Identify and prioritise core feature slices with key stakeholders
- Break down and explain the theory behind each stage in the strategy:
  - how each step would be implemented in practice (e.g. there will be some timeline overlap between phases 3 and 4 as features are delivered in full-stack feature slices)
  - methodology used to "10x" development speed and scale a team working on a software project (e.g. discussions about software quality and scaling)
- Plan feature breakdown of releasable increments in line with priorities and key business dates.

📚 Resources & References

🏃 **Delivery and Methodology:**

- Break apart your features into full-stack slices
- Lean Project and Portfolio Management Framework

▼ The Lean Tech Manifesto: Learn the Secrets of Tech Leaders to Grasp the Full Benefits of Agile at Scale

  - Explains why Agile was never intended to scale and how Lean offers a solution to scaling tech organisations
  - SAFe is "Shitty Agile For Enterprises" - Martin Fowler, signatory of the Agile Manifesto

- The Misunderstanding of Andon
- The Lean metaphor of the rocks in the lake

⚙️ **Tech and Software Craft:**

- Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations

- Surviving continuous deployment in distributed systems
- Is High Quality Software Worth the Cost?
- How To Not Die By A Thousand Cuts. Or, How To Think About Software Quality.
- The History of Pets vs Cattle and How to Use the Analogy Properly